

Relasi Database Tingkat Lanjut: Mengelola Relasi Tabel, Advanced Query, Stored Procedure, dan Audit Trail di PostgreSQL

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Sebagai Backend Engineer Go, kemampuan mengelola database tidak hanya terbatas pada operasi CRUD sederhana. Anda harus mampu merancang skema relasi yang efisien, mengoptimalkan kueri kompleks menggunakan fitur mutakhir database, serta mengotomatiskan logika bisnis dan pencatatan riwayat aktivitas langsung di dalam database menggunakan *Stored Procedure*, *Trigger*, dan *Audit Trail*.

Modul ini akan membahas relasi tabel beserta integritas referensialnya, teknik kueri tingkat lanjut seperti CTE (Common Table Expressions) dan Window Functions, perbedaan mendasar antara Function dan Stored Procedure, serta cara mengotomatiskan pembuatan log perubahan data (*Audit Trail*) berbasis trigger.

Sesi 1: Relasi Tabel & Integritas Referensial (45 Menit)

Model data relasional bergantung pada keterhubungan (*relationship*) antar-tabel untuk mencerminkan kondisi dunia nyata. PostgreSQL menyediakan mekanisme penegakan integritas data yang sangat ketat melalui *Foreign Key Constraints*.

1. Hubungan One-to-One (Satu ke Satu)

Relasi *One-to-One* terjadi jika setiap baris data di Tabel A hanya terhubung dengan maksimal satu baris di Tabel B. Relasi ini dipaksakan dengan cara memberikan constraint `UNIQUE` pada kolom *Foreign Key*.

- **Contoh Kasus:** Tabel `users` dengan `user_profiles`.
- **Implementasi SQL:**

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE user_profiles (  
  user_id SERIAL PRIMARY KEY,  
  profile_data TEXT,
```

```

id SERIAL PRIMARY KEY,
user_id INT UNIQUE NOT NULL, -- UNIQUE memastikan One-to-One
full_name VARCHAR(100) NOT NULL,
phone_number VARCHAR(15),
address TEXT,
CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

2. Hubungan One-to-Many (Satu ke Banyak)

Relasi *One-to-Many* adalah jenis relasi yang paling sering digunakan, di mana satu baris data di Tabel A dapat terhubung ke beberapa baris data di Tabel B.

- **Contoh Kasus:** Tabel `categories` dengan `products`. Satu kategori memiliki banyak produk.
- **Implementasi SQL:**

```

CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  category_id INT NOT NULL,
  sku VARCHAR(50) UNIQUE NOT NULL,
  name VARCHAR(100) NOT NULL,
  price NUMERIC(12, 2) NOT NULL,
  stock INT NOT NULL DEFAULT 0,
  CONSTRAINT fk_category FOREIGN KEY (category_id) REFERENCES categories(id) ON
DELETE RESTRICT
);

```

3. Hubungan Many-to-Many (Banyak ke Banyak)

Relasi *Many-to-Many* terjadi saat beberapa baris data di Tabel A dapat terhubung ke beberapa baris data di Tabel B. Untuk menerapkannya, kita memerlukan tabel perantara (*junction/associative table*) yang menyimpan pasangan kunci asing (*foreign keys*) dari kedua tabel utama.

- **Contoh Kasus:** Tabel `orders` dengan `products`. Satu pesanan bisa memuat banyak produk, dan satu produk bisa dibeli dalam banyak pesanan.
- **Implementasi SQL:**

```

CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  order_number VARCHAR(50) UNIQUE NOT NULL,
  customer_name VARCHAR(100) NOT NULL,
  total_amount NUMERIC(12, 2) NOT NULL,

```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- Tabel Junction/Associative Table
CREATE TABLE order_items (
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    price NUMERIC(12, 2) NOT NULL,
    PRIMARY KEY (order_id, product_id), -- Composite Primary Key
    CONSTRAINT fk_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE
CASCADE,
    CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE
RESTRICT
);

```

4. Referential Actions (ON DELETE / ON UPDATE)

Ketika baris data di tabel induk dihapus atau diperbarui, PostgreSQL memungkinkan Anda menentukan aksi otomatis pada data anak:

- **ON DELETE CASCADE** : Menghapus baris di tabel anak secara otomatis jika baris induknya dihapus.
- **ON DELETE RESTRICT** : Mencegah penghapusan baris induk jika baris anak masih merujuk padanya (ini adalah aksi *default* yang paling aman).
- **ON DELETE SET NULL** : Mengubah nilai kolom foreign key di tabel anak menjadi **NULL** jika baris induk dihapus.

Sesi 2: Advanced Querying: CTE dan Window Functions (45 Menit)

Dalam aplikasi e-commerce berskala besar, menyusun laporan penjualan membutuhkan penulisan kueri SQL yang kompleks. PostgreSQL memiliki fitur handal untuk menyederhanakan dan mengoptimalkan kueri tersebut.

1. CTE (Common Table Expressions)

CTE digunakan untuk menyusun subquery menjadi blok-blok terpisah yang lebih bersih dan mudah dibaca menggunakan klausa **WITH**. CTE bertindak layaknya tabel temporer dalam cakupan eksekusi kueri tersebut.

- **Kasus:** Mencari kategori produk yang memiliki total penjualan lebih tinggi dari rata-rata penjualan semua kategori.

```

WITH CategorySales AS (
  -- Menghitung total penjualan per kategori
  SELECT
    c.name AS category_name,
    SUM(oi.quantity * oi.price) AS total_sales
  FROM order_items oi
  JOIN products p ON oi.product_id = p.id
  JOIN categories c ON p.category_id = c.id
  GROUP BY c.name
),
AverageSales AS (
  -- Menghitung rata-rata penjualan dari seluruh kategori
  SELECT AVG(total_sales) AS avg_sales FROM CategorySales
)
SELECT cs.category_name, cs.total_sales
FROM CategorySales cs, AverageSales av
WHERE cs.total_sales > av.avg_sales;

```

2. Window Functions

Window Function memungkinkan Anda melakukan kalkulasi kumulatif atau analitis lintas baris data tanpa menggabungkan baris tersebut menjadi satu baris tunggal (berbeda dengan `GROUP BY`). Fungsi window ditulis menggunakan klausa `OVER (PARTITION BY ... ORDER BY ...)`.

- **Fungsi Window yang Sering Digunakan:**
 - `ROW_NUMBER()` : Memberikan nomor urut baris yang unik secara berurutan.
 - `RANK()` : Memberikan peringkat dengan nilai lompat jika ada data kembar.
 - `DENSE_RANK()` : Memberikan peringkat tanpa lompatan nilai jika ada data kembar.
 - `SUM(...) OVER(...)` : Menghitung nilai akumulasi berjalan (*running total*).
- **Kasus:** Mendapatkan peringkat produk terlaris di masing-masing kategori produk berdasarkan jumlah kuantitas yang terjual.

```

WITH ProductPopularity AS (
  SELECT
    c.name AS category_name,
    p.name AS product_name,
    SUM(oi.quantity) AS total_qty
  FROM order_items oi
  JOIN products p ON oi.product_id = p.id
  JOIN categories c ON p.category_id = c.id
  GROUP BY c.name, p.name
)
SELECT
  category_name,
  product_name,
  total_qty,
  DENSE_RANK() OVER (

```

```
    PARTITION BY category_name
    ORDER BY total_qty DESC
) as rank_in_category
FROM ProductPopularity;
```

Sesi 3: Stored Procedure & Trigger (45 Menit)

Untuk menjaga konsistensi data dan mengurangi komunikasi bolak-balik (*network round-trip*) antara server Go dan database, kita dapat menulis logika pemrograman prosedural langsung di database menggunakan PL/pgSQL.

1. Perbedaan Function vs Stored Procedure

PostgreSQL menyediakan dua objek pemrograman prosedural utama:

Fitur	Function (CREATE FUNCTION)	Stored Procedure (CREATE PROCEDURE)
Eksekusi	Dipanggil sebagai bagian dari query (SELECT)	Dipanggil menggunakan perintah CALL
Nilai Kembali	Harus mengembalikan nilai (RETURNS <type>)	Tidak wajib mengembalikan nilai
Manajemen Transaksi	Tidak dapat mematikan/menyalakan transaksi	Mendukung COMMIT dan ROLLBACK di dalamnya
Tujuan Utama	Perhitungan data, transformasi nilai	Logika bisnis yang melibatkan manipulasi data masal

- Contoh Stored Procedure dengan Manajemen Transaksi:

```
CREATE OR REPLACE PROCEDURE insert_bulk_categories(cat_name_1 VARCHAR, cat_name_2
VARCHAR)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO categories (name) VALUES (cat_name_1);
    INSERT INTO categories (name) VALUES (cat_name_2);

    -- Melakukan commit langsung di tengah proses eksekusi
    COMMIT;
EXCEPTION
```

```
WHEN OTHERS THEN
    -- Melakukan rollback total jika terjadi kegagalan
    ROLLBACK;
    RAISE NOTICE 'Terjadi error, transaksi dibatalkan!';
END;
$$;
```

2. Triggers (Logika Database Berbasis Event)

Trigger adalah sebuah blok perintah yang dipicu secara otomatis oleh database sebagai reaksi atas terjadinya *event* berupa `INSERT`, `UPDATE`, atau `DELETE` pada tabel tertentu.

- **Mekanisme Eksekusi Trigger:**

- `BEFORE` : Dieksekusi sebelum data ditulis ke disk (berguna untuk validasi data).
- `AFTER` : Dieksekusi setelah data ditulis ke disk (berguna untuk pencatatan riwayat audit atau sinkronisasi).
- `FOR EACH ROW` : Dieksekusi untuk setiap baris data yang terpengaruh (memiliki akses ke variabel `NEW` dan `OLD`).
- `FOR EACH STATEMENT` : Dieksekusi sekali saja untuk satu perintah SQL penuh.

Di PostgreSQL, pembuatan trigger terdiri dari 2 langkah:

1. Membuat **Trigger Function** (fungsi tanpa parameter yang mengembalikan tipe data `TRIGGER`).
2. Mengikat fungsi tersebut ke tabel target menggunakan perintah `CREATE TRIGGER`.

Sesi 4: Praktikum Mandiri: Database E-Commerce dengan Audit Trail & Logs (45 Menit)

Sekarang kita akan merancang sistem e-commerce mini di PostgreSQL. Praktikum ini mencakup pembuatan skema relasi, pembuatan Stored Procedure untuk checkout yang aman, serta implementasi sistem *Audit Trail* berbasis trigger untuk mencatat riwayat perubahan data pada tabel produk.

1. Desain Skema Database

Jalankan skrip berikut untuk membangun tabel dan relasi dasar:

```
-- Hapus tabel lama jika ada
DROP TABLE IF EXISTS audit_logs CASCADE;
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS products CASCADE;
```

```

DROP TABLE IF EXISTS categories CASCADE;

-- Tabel Kategori
CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL
);

-- Tabel Produk
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    category_id INT NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
    sku VARCHAR(50) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    price NUMERIC(12, 2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabel Pesanan
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    order_number VARCHAR(50) UNIQUE NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    total_amount NUMERIC(12, 2) NOT NULL DEFAULT 0,
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Detail Pesanan (Junction Table)
CREATE TABLE order_items (
    order_id INT REFERENCES orders(id) ON DELETE CASCADE,
    product_id INT REFERENCES products(id) ON DELETE RESTRICT,
    quantity INT NOT NULL CHECK (quantity > 0),
    price NUMERIC(12, 2) NOT NULL,
    PRIMARY KEY (order_id, product_id)
);

-- Tabel Audit Trail (Log Perubahan Data)
CREATE TABLE audit_logs (
    id SERIAL PRIMARY KEY,
    table_name VARCHAR(50) NOT NULL,
    action_type VARCHAR(10) NOT NULL, -- INSERT, UPDATE, DELETE
    record_id INT NOT NULL,
    old_data JSONB, -- Menyimpan data lama dalam format JSON
    new_data JSONB, -- Menyimpan data baru dalam format JSON
    changed_by VARCHAR(50) DEFAULT CURRENT_USER,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

2. Implementasi Stored Procedure untuk Pemesanan (Checkout)

Procedure ini digunakan untuk memproses pesanan barang secara aman. Prosedur ini akan memeriksa ketersediaan stok produk, mengurangi stok produk, membuat baris pesanan, memasukkan baris detail pesanan, dan melakukan transaksi secara atomik.

```
CREATE OR REPLACE PROCEDURE process_checkout(
    p_customer_name VARCHAR,
    p_product_id INT,
    p_quantity INT,
    OUT p_order_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_sku VARCHAR(50);
    v_price NUMERIC(12, 2);
    v_stock INT;
    v_order_number VARCHAR(50);
    v_total NUMERIC(12,2);
BEGIN
    -- 1. Mengambil informasi detail produk sekaligus mengunci baris produk (Row-Level Locking)
    SELECT sku, price, stock INTO v_sku, v_price, v_stock
    FROM products
    WHERE id = p_product_id
    FOR UPDATE; -- Mengunci baris produk dari transaksi lain selama proses ini berjalan

    -- Validasi keberadaan produk
    IF v_sku IS NULL THEN
        RAISE EXCEPTION 'Checkout gagal: Produk ID % tidak ditemukan', p_product_id;
    END IF;

    -- Validasi kecukupan stok
    IF v_stock < p_quantity THEN
        RAISE EXCEPTION 'Checkout gagal: Stok % tidak mencukupi. Sisa stok: %, Permintaan: %', v_sku, v_stock, p_quantity;
    END IF;

    -- 2. Membuat nomor pesanan unik
    v_order_number := 'ORD-' || TO_CHAR(NOW(), 'YYYYMMDD') || '-' ||
    LPAD(FLOOR(RANDOM() * 10000)::TEXT, 4, '0');
    v_total := v_price * p_quantity;

    -- 3. Membuat pesanan baru
    INSERT INTO orders (order_number, customer_name, total_amount, status)
    VALUES (v_order_number, p_customer_name, v_total, 'COMPLETED')
    RETURNING id INTO p_order_id;

    -- 4. Memasukkan item ke dalam tabel detail pesanan
```



```

INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES (p_order_id, p_product_id, p_quantity, v_price);

-- 5. Mengurangi stok produk di database
UPDATE products
SET stock = stock - p_quantity
WHERE id = p_product_id;

-- Melakukan Commit Transaksi secara eksplisit
COMMIT;
RAISE NOTICE 'Checkout Berhasil! ID Pesanan: %, Total: Rp %', p_order_id, v_total;

EXCEPTION
    WHEN OTHERS THEN
        -- Melakukan Rollback jika terjadi kesalahan apa pun
        ROLLBACK;
        RAISE EXCEPTION 'Transaksi checkout dibatalkan akibat kesalahan: %', SQLERRM;
END;
$$;

```

3. Implementasi Trigger Audit Trail

Kini kita akan membuat sistem pencatatan otomatis (*Audit Trail*) untuk mendeteksi setiap perubahan data (tambah, ubah, hapus) di tabel `products`.

```

-- 1. Membuat Trigger Function
CREATE OR REPLACE FUNCTION fn_audit_product_changes()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF (TG_OP = 'INSERT') THEN
        INSERT INTO audit_logs (table_name, action_type, record_id, old_data,
new_data)
        VALUES ('products', TG_OP, NEW.id, NULL, to_jsonb(NEW));
        RETURN NEW;
    ELSIF (TG_OP = 'UPDATE') THEN
        INSERT INTO audit_logs (table_name, action_type, record_id, old_data,
new_data)
        VALUES ('products', TG_OP, NEW.id, to_jsonb(OLD), to_jsonb(NEW));
        RETURN NEW;
    ELSIF (TG_OP = 'DELETE') THEN
        INSERT INTO audit_logs (table_name, action_type, record_id, old_data,
new_data)
        VALUES ('products', TG_OP, OLD.id, to_jsonb(OLD), NULL);
        RETURN OLD;
    END IF;
    RETURN NULL;
END;
$$;

```

```

-- 2. Mengikat Trigger ke Tabel products
CREATE TRIGGER tr_audit_products
AFTER INSERT OR UPDATE OR DELETE
ON products
FOR EACH ROW
EXECUTE FUNCTION fn_audit_product_changes();

```

4. Skenario Pengujian Logika Database

Mari kita uji seluruh fungsionalitas di atas dengan menjalankan beberapa query simulasi berikut:

```

-- 1. Tambah Data Master Kategori
INSERT INTO categories (name) VALUES ('Elektronik'), ('Pakaian');

-- 2. Tambah Data Master Produk (Ini akan memicu trigger audit INSERT)
INSERT INTO products (category_id, sku, name, price, stock)
VALUES
(1, 'SKU-001', 'Keyboard Mechanical', 750000.00, 50),
(1, 'SKU-002', 'Mouse Gaming Wireless', 450000.00, 30);

-- Periksa isi tabel audit trail setelah insert produk
SELECT * FROM audit_logs;

-- 3. Ubah Harga Produk (Ini akan memicu trigger audit UPDATE)
UPDATE products
SET price = 800000.00, stock = 48
WHERE sku = 'SKU-001';

-- Periksa isi tabel audit trail setelah update produk
SELECT * FROM audit_logs WHERE action_type = 'UPDATE';

-- 4. Jalankan Checkout via Stored Procedure
DO $$
DECLARE
    v_id INT;
BEGIN
    -- Memanggil procedure dengan parameter customer, product_id (1), qty (5)
    CALL process_checkout('Budi Santoso', 1, 5, v_id);
END;
$$;

-- Periksa apakah stok berkurang dan pesanan terbentuk
SELECT * FROM products WHERE id = 1;
SELECT * FROM orders;
SELECT * FROM order_items;
SELECT * FROM audit_logs WHERE action_type = 'UPDATE'; -- Perubahan stok tercatat di
audit trail!

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemeliharaan database di bawah ini untuk memperkuat pemahaman Anda tentang integrasi kueri analitik, stored procedure, dan otomatisasi trigger.

Latihan 1: Log Audit Mutasi Stok Barang Terperinci (Stock Ledger)

Deskripsi Soal: Sistem e-commerce yang baik membutuhkan histori mutasi stok yang sangat terperinci (tidak hanya sekadar mencatat nilai lama dan baru dalam format JSON umum, tetapi mencatat alasan mutasi seperti "CHECKOUT", "PENGEMBALIAN", "RESTOCK", dll).

Kriteria Implementasi:

1. Buatlah tabel bernama `stock_mutations` dengan kolom:
 - `id` (SERIAL PRIMARY KEY)
 - `product_id` (INT, Foreign Key ke `products`)
 - `old_stock` (INT)
 - `new_stock` (INT)
 - `quantity_changed` (INT, bisa bernilai negatif untuk pengurangan dan positif untuk penambahan)
 - `mutation_reason` (VARCHAR, nilai default: 'SYSTEM_UPDATE')
 - `created_at` (TIMESTAMP)
2. Buatlah trigger function `fn_log_stock_mutations` pada tabel `products` yang terpicu **hanya ketika kolom `stock` mengalami perubahan (UPDATE)**.

TIP

Gunakan kondisi filter trigger `WHEN (OLD.stock IS DISTINCT FROM NEW.stock)` saat mengikat trigger agar proses audit tidak terpicu jika kolom harga atau nama produk yang di-update.

3. Di dalam fungsi trigger tersebut, tentukan logika pengisian kolom `mutation_reason` :
 - Jika stok berkurang, asumsikan alasan mutasi adalah `'STOCK_DEDUCTION'` .
 - Jika stok bertambah, asumsikan alasan mutasi adalah `'STOCK_RESTOCK'` .
4. Uji trigger tersebut dengan melakukan update stok manual pada salah satu produk, kemudian tampilkan isi tabel `stock_mutations` .

Latihan 2: Stored Procedure untuk Pembatalan Pesanan (`cancel_order`)

Deskripsi Soal: Ketika pembeli membatalkan pesanan e-commerce, sistem harus mengembalikan stok barang yang dibeli ke kondisi semula (*restock*) secara otomatis dan memperbarui status pesanan menjadi `'CANCELLED'` .

Kriteria Implementasi:

1. Buatlah sebuah Stored Procedure bernama `cancel_order(p_order_id INT)` .
2. Di dalam procedure tersebut, implementasikan langkah-masing berikut dalam cakupan transaksi tunggal:
 - Periksa apakah pesanan dengan ID tersebut ada. Jika tidak, lemparkan eksepsi error.
 - Periksa status pesanan saat ini. Jika status pesanan sudah `'CANCELLED'` , lemparkan eksepsi error (mencegah pembatalan ganda).
 - Lakukan perulangan (*looping*) melalui cursor/select data dari `order_items` yang terkait dengan `p_order_id` tersebut.
 - Untuk setiap produk yang dibeli di pesanan tersebut, lakukan update untuk menambahkan kembali kuantitas yang dibeli ke kolom `stock` di tabel `products` .
 - Ubah status pesanan di tabel `orders` menjadi `'CANCELLED'` .
3. Gunakan blok `EXCEPTION` untuk menangkap kesalahan tak terduga dan memicu `ROLLBACK` demi menjaga integritas data.
4. Panggil stored procedure tersebut pada pesanan yang sudah dibuat di Sesi 4, lalu pastikan stok produk terkait kembali bertambah dan log mutasi stok (dari Latihan 1) otomatis terisi.